

Challenge Tecnológico Integrador: "GuardiánClima ITBA"

¡Bienvenidos al Challenge Tecnológico "GuardiánClima ITBA"! Este desafío está diseñado para que, trabajando en equipo, desarrollen una aplicación de consola en Python que les permita interactuar con datos climáticos, implementar mecanismos de validación de contraseñas (incluyendo guía hacia opciones más seguras), analizar datos globales de uso y utilizar inteligencia artificial.

Objetivo General

Desarrollar una aplicación de consola en Python llamada "GuardiánClima ITBA". La aplicación presentará un menú inicial para iniciar sesión o registrar nuevos usuarios. El proceso de registro pondrá un fuerte énfasis en la validación de contraseñas según criterios de seguridad establecidos, y guiará al usuario hacia la creación de opciones más seguras si su intento inicial no es suficiente. Una vez autenticado, el usuario accederá a un menú principal para consultar el clima, guardar un historial global de consultas, generar estadísticas globales y ofrecer consejos de vestimenta mediante IA. El proyecto debe integrar de manera demostrable conceptos de Programación, Ciberseguridad, Análisis de Datos, Inteligencia Artificial, y Cloud Computing y Conectividad, los cuales deberán ser explicados y evidenciados a lo largo de la documentación del proyecto.

Flujo de la Aplicación y Gestión de Usuarios

La aplicación comenzará con un Menú de Acceso. Solo después de un inicio de sesión o registro exitoso se accederá al Menú Principal.

Importante Aclaración de Simulación: Para este ejercicio, la gestión de usuarios (almacenamiento y login) es una simulación con fines educativos y NO es segura para aplicaciones reales. El objetivo principal es que entiendan lo que es una contraseña segura. El almacenamiento de contraseñas en usuarios_simulados.csv (como se describe a continuación) es inherentemente inseguro si las contraseñas están en texto plano o de forma fácilmente reversible, y solo se utilizará aquí para simplificar la simulación del acceso y registro.

Menú de Acceso (Pre-Login)

Al iniciar la aplicación, se presentará el siguiente menú:

1. Iniciar Sesión:
 - Solicitar nombre de usuario y contraseña.
 - Leer el archivo usuarios_simulados.csv (que almacena username,password_simulada).
 - Verificar las credenciales.

- Si la autenticación es exitosa, proceder al Menú Principal.
- Si falla, mostrar un mensaje y permitir reintentar o volver al Menú de Acceso.

2. Registrar Nuevo Usuario:

- Solicitar un nuevo nombre de usuario. La aplicación verificará (leyendo usuarios_simulados.csv) si el nombre de usuario ya existe. Si existe, informar y pedir otro.
- Solicitar una contraseña. Esta contraseña debe ser validada por una función que implementen, asegurando que cumpla con al menos 3 criterios mínimos de seguridad vistos en la materia a elegir por el grupo.
- Si la contraseña no cumple con alguno de estos criterios, la aplicación debe rechazarla e informar al usuario específicamente qué regla(s) no cumplió. Además, el sistema deberá sugerir cómo crear una opción más robusta, por ejemplo:

"Tu contraseña no cumple con [listar reglas incumplidas]. Para una contraseña más segura, considera usar [recomendaciones]."

- Si la contraseña es aceptada, se guarda el nombre de usuario y la contraseña (que pasó la validación) en el archivo usuarios_simulados.csv.
- Tras un registro exitoso, se sugiere que la aplicación lleve al usuario directamente al Menú Principal (como si se hubiera auto-logueado) para una mejor experiencia de usuario.

3. Salir de la Aplicación:

- Terminar la ejecución del programa.

Menú Principal de GuardiánClima ITBA (Post-Login)

Una vez "autenticado" a través del Menú de Acceso, el usuario tendrá acceso al siguiente menú:

1. Opción 1: Consultar Clima Actual y Guardar en Historial Global

- Solicitar al usuario el nombre de una ciudad.
- Conectarse a una API de clima sugerida (ver Anexo: Uso de APIs para detalles sobre OpenWeatherMap) para obtener datos actuales (temperatura, sensación térmica, humedad, descripción del clima, velocidad del viento, etc.).

- Mostrar la información de forma clara.
- Guardar la consulta (incluyendo NombreDeUsuario del usuario logueado, Ciudad, Fecha/Hora completa, Temperatura_C, Condicion_Clima, Humedad_Porcentaje, Viento_kmh) en un único archivo de historial global CSV llamado historial_global.csv. Cada nueva consulta de cualquier usuario se añadirá como una nueva fila en este archivo.

2. Opción 2: Ver Mi Historial Personal de Consultas por Ciudad

- Solicitar al usuario el nombre de una ciudad.
- Leer el archivo historial_global.csv.
- Filtrar y mostrar todas las entradas que correspondan al nombre de usuario del usuario logueado Y a la ciudad especificada. Presentar los datos de forma ordenada (ej. Fecha, Hora, Temperatura, Condición).

3. Opción 3: Estadísticas Globales de Uso y Exportar Historial Completo

- Procesar el archivo historial_global.csv para calcular y mostrar las siguientes estadísticas sobre el total de datos de todos los usuarios:
 - Ciudad más consultada (considerando todas las consultas en historial_global.csv).
 - Número total de consultas realizadas por todos los usuarios.
 - Temperatura promedio registrada en todas las consultas de todos los usuarios.
- La aplicación se asegurará de que el archivo historial_global.csv esté actualizado con todas las consultas. Este archivo es el que los estudiantes podrán usar luego (fuera de la aplicación, con herramientas como Excel, Google Sheets) para crear los siguientes tipos de gráficos basados en los datos globales:
 - Gráfico de Barras: Para visualizar, por ejemplo, el número de consultas por cada ciudad (datos de todos los usuarios).
 - Gráfico de Líneas: Para visualizar, por ejemplo, la tendencia de la temperatura para una ciudad específica a lo largo del tiempo, considerando todas las consultas registradas para esa ciudad.

- Gráfico de Torta (Pie Chart): Para visualizar, por ejemplo, la distribución porcentual de las diferentes condiciones climáticas (Soleado, Nublado, Lluvioso, etc.) registradas en todo el historial global.

4. Opción 4: Consejo IA: ¿Cómo Me Visto Hoy? 🧑🤖

- Utilizar los datos de la última consulta de clima realizada por el usuario (o permitirle seleccionar una de su historial, o realizar una nueva consulta específica para esta función).
- Conectarse a una API de Inteligencia Artificial generativa como Google Gemini (ver Anexo: Uso de APIs para una guía sobre cómo usar Gemini).
- El equipo deberá diseñar un prompt efectivo que, al enviar los parámetros climáticos relevantes a la API de IA, esta devuelva un consejo breve y práctico sobre cómo vestirse adecuado al clima.
- Mostrar el consejo generado por la IA al usuario.

5. Opción 5: Acerca De...

- Mostrar una descripción detallada de la aplicación "GuardiánClima ITBA".
- Explicar cómo se usa cada opción del menú, incluyendo el flujo del Menú de Acceso.
- Describir brevemente cómo funciona internamente, incluyendo:
 - El proceso simulado de creación de usuarios, la importancia de la validación de contraseñas según los criterios establecidos, y la guía hacia opciones más robustas.
 - La advertencia sobre el almacenamiento simulado e inseguro de credenciales usado en este ejercicio y la mención de técnicas seguras como el hashing.
 - Obtención de datos de clima y guardado de historial global.
 - Generación de estadísticas globales y preparación del CSV para gráficos.
 - Interacción con la IA para consejos.
- Listar los nombres de los desarrolladores (el equipo).
- Elegir un nombre para el grupo y mostrarlo en esta sección

6. Opción 6: Cerrar Sesión

- Finaliza la sesión del usuario actual.
- La aplicación regresa al Menú de Acceso.

Stack Tecnológico y Herramientas Sugeridas

- Lenguaje: Python 3.x
- Gestión Simulada de Usuarios e Historial Global:
 - Manejo de archivos de texto (CSV) para usuarios_simulados.csv y historial_global.csv.
 - Lógica de validación de contraseñas implementada en Python.
- APIs Externas:
 - API de Clima: OpenWeatherMap (gratuita con API key).
 - API de IA: Google Gemini API (accesible vía Google AI Studio).
- Manejo de Peticiones HTTP: Librería requests.
- Cliente API de IA: Librería google-generativeai (para Gemini).
- Manejo de CSV: Módulo csv de Python.

Entregables

1. Código Fuente Completo:

- Todos los archivos .py bien comentados y estructurados.
- Un archivo README.md con:
 - Instrucciones claras para configurar el entorno (instalación de librerías: requests, google-generativeai).
 - Instrucciones para ejecutar la aplicación, detallando el flujo de menús.
 - Información sobre cómo y dónde configurar las API Keys necesarias (Weather, Gemini) de forma segura (sin incluir las keys en el repositorio).

2. Documento del Proyecto (Formato PDF, breve):

- Introducción: Descripción del proyecto, objetivos.
- Diseño y Arquitectura: Cómo está estructurada la aplicación, flujo de datos (incluyendo el flujo de menús), decisiones de diseño.

- Guía de Usuario Detallada: Cómo usar cada funcionalidad del Menú de Acceso y del Menú Principal.
 - Desafíos y Soluciones: Problemas encontrados durante el desarrollo y cómo los resolvieron.
 - Conclusiones y Aprendizajes del Equipo.
3. Gráficos Excel generados
- Archivo Excel con los gráficos de barra, línea y torta.
4. Video Demo (Máximo 5-7 minutos):
- Demostración de la aplicación: comenzando por el Menú de Acceso, el proceso de registro (mostrando rechazo de contraseñas débiles, la sugerencia de opción robusta, y aceptación de una fuerte), el login, la navegación al Menú Principal, el uso de las opciones principales (consulta de clima, historial, estadísticas globales en consola, consejo IA), mostrando los archivos CSV generados, y finalmente la opción de Cerrar Sesión.

Consideraciones Adicionales

- Manejo de Errores: La aplicación debe ser robusta y manejar posibles errores de forma elegante (ej. API no disponible, archivo no encontrado, entrada incorrecta del usuario, ciudad no encontrada, etc.).
 - Modularidad: Organicen el código en funciones y, si aplica, en módulos para mayor claridad y reutilización.
 - Usabilidad: Presten atención a que la interfaz de consola sea clara y fácil de usar para el usuario.
-

Anexo: Uso de APIs – OpenWeatherMap y Google Gemini

Este anexo proporciona una guía básica para interactuar con las APIs sugeridas para el proyecto.

Parte 1: API de Clima (OpenWeatherMap)

¿Qué es una API (Application Programming Interface)?

Imagina que estás en un restaurante. Tú (tu aplicación) quieres pedir comida (datos o funcionalidades específicas). No vas directamente a la cocina (el servidor o la base de datos del proveedor de servicios), sino que¹ hablas con un mesero (la API). Una API es un conjunto de reglas y protocolos que permite que diferentes aplicaciones de software se comuniquen e intercambien información. Tu aplicación (el "cliente") hace una "petición" y el servidor de la API envía una "respuesta".

Usando la API de OpenWeatherMap en Python:

1. Registrarse y Obtener una API Key:

- Ve a [OpenWeatherMap](#) y regístrate.
- Encuentra tu API Key en la sección "API keys" de tu perfil.
- ¡Importante! Trata tu API Key como una contraseña; no la compartas públicamente ni la subas a repositorios.

2. Consultar la Documentación y Construir la URL:

- El endpoint para clima actual es aproximadamente `https://api.openweathermap.org/data/2.5/weather`.
- Parámetros comunes: q (ciudad),appid (tu API Key), units=metric (para Celsius), lang=es (para español).
- Ejemplo URL:
`https://api.openweathermap.org/data/2.5/weather?q=BuenosAires&appid=TU_API_KEY&units=metric&lang=es`

3. Realizar la Petición y Procesar la Respuesta en Python:

```
import requests
```

```
import json
```

```
def obtener_clima_ciudad_owm(ciudad, api_key):
```

```
    base_url = "https://api.openweathermap.org/data/2.5/weather"
```

```

parametros = {
    'q': ciudad,
    'appid': api_key,
    'units': 'metric',
    'lang': 'es'
}

print(f"\nConsultando el clima (OpenWeatherMap) para: {ciudad}...")

try:
    respuesta = requests.get(base_url, params=parametros, timeout=10)
    respuesta.raise_for_status()
    datos_clima = respuesta.json()
    return datos_clima
except requests.exceptions.HTTPError as errh:
    if respuesta.status_code == 401:
        print(f"Error de autenticación OWM: API Key inválida.")
    elif respuesta.status_code == 404:
        print(f"Error OWM: Ciudad '{ciudad}' no encontrada.")
    else:
        print(f"Error HTTP OWM: {errh}")
    return None
except requests.exceptions.RequestException as err:
    print(f"Error de conexión/petición OWM: {err}")
    return None
except json.JSONDecodeError:
    print(f"Error OWM: La respuesta de la API no es JSON válido.")
    return None

```

--- Ejemplo de cómo extraer datos (dentro de tu lógica de Opción 1) ---


```
# mi_clave_owm = "TU_OPENWEATHERMAP_API_KEY"

# datos_clima_recibidos = obtener_clima_ciudad_owm("Buenos Aires",
mi_clave_owm)

# if datos_clima_recibidos:

#     try:

#         temperatura = datos_clima_recibidos['main']['temp']

#         descripcion = datos_clima_recibidos['weather'][0]['description']

#         # ... extraer más datos según necesidad ...

#         print(f"Temperatura en Buenos Aires: {temperatura}°C,
{descripcion.capitalize()}")

#     except KeyError:

#         print("Error: Formato inesperado en datos de OWM.")
```

Parte 2: API de Inteligencia Artificial (Google Gemini)

Para la Opción 4 (Consejo de Vestimenta), pueden usar la API de Google Gemini.

1. Obtener una API Key para Gemini:
 - Visiten [Google AI Studio](#).
 - Necesitarán una cuenta de Google. Desde AI Studio, pueden crear un proyecto y generar una API key.
 - ¡Importante! Esta API key también es sensible y debe protegerse.
2. Instalar la Librería de Python:
 - Abran una terminal o command prompt y ejecuten: `pip install google-generativeai`
3. Usar la API de Gemini en Python para generar texto:

```
import google.generativeai as genai

def obtener_consejo_ia_gemini(api_key_gemini, temperatura, condicion_clima,
viento, humedad):

    """

    Obtiene un consejo de vestimenta usando la API de Gemini.

    """

    try:
```

```

genai.configure(api_key=api_key_gemini)

# Configuración del modelo (pueden explorar diferentes modelos si lo
deseen)

# Para generación de texto simple, 'gemini-pro' suele ser una buena opción.
model = genai.GenerativeModel('gemini-pro')

# Diseñar el prompt (este es un ejemplo, ustedes deben crear el suyo)
# Es importante dar contexto y ser específico para obtener mejores resultados.
prompt_diseñado_por_equipo = (
    f"PROMPT"
)

    print("\nGenerando consejo de vestimenta con IA...")

# Generar contenido
response = model.generate_content(prompt_diseñado_por_equipo)

# Asegurarse de que hay texto en la respuesta
if response.text:

    return response.text

else:

    # A veces la API puede no devolver texto si hay problemas con el prompt o
filtros de seguridad

    # Investigar response.prompt_feedback si hay problemas

    print("La IA no pudo generar un consejo. Razón (si está disponible):",
response.prompt_feedback)

    return "No se pudo generar un consejo en este momento."

except Exception as e:

    print(f"Error al contactar la API de Gemini o procesar la respuesta: {e}")

    return "Error al generar el consejo de IA."

```